

Absolutely. We have enough requirements to design the framework. I'll preserve your choices exactly and won't introduce unrequested features such as parallel execution, Faker, Allure, or additional configuration mechanisms.

1. Finalized Requirements

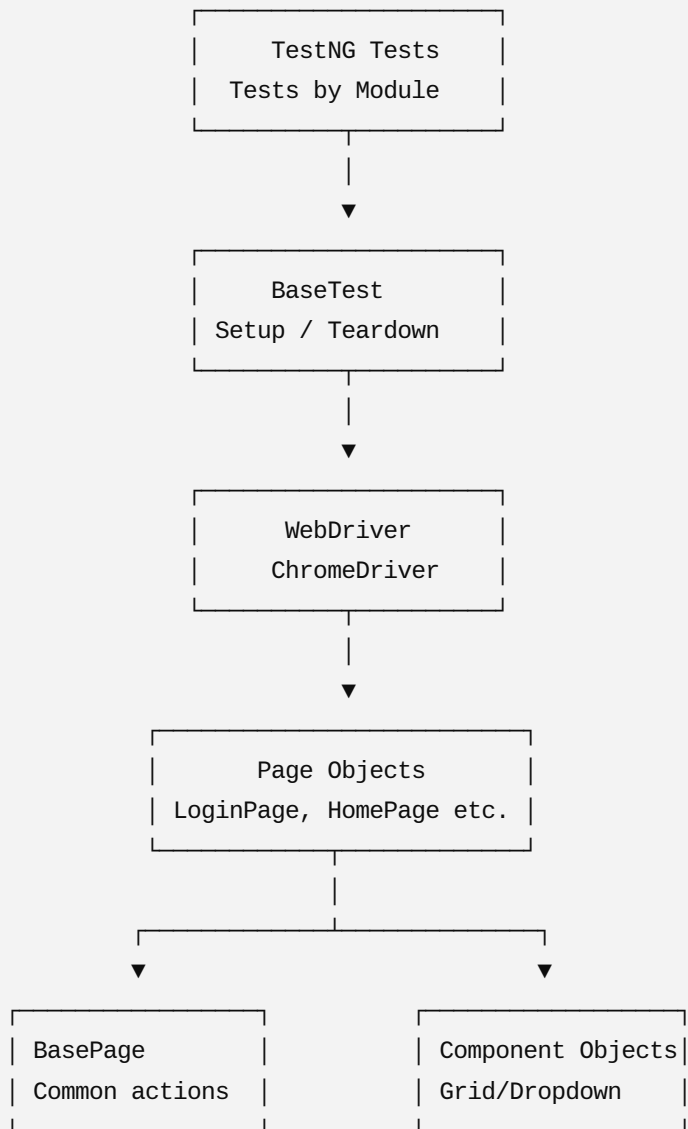
Area	Your choice
Application	Web application
Browser	Chrome
Test framework	TestNG
Build tool	Maven
Language	Java
Design pattern	Page Object Model
Page architecture	Base Page + Page Objects + Component Objects
WebDriver lifecycle	New driver per test method
Driver creation	Directly in <code>BaseTest</code>
Wait strategy	Explicit waits only
Test data	Hybrid
Business test data	Excel
Configuration	<code>config.properties</code>
Sensitive data	<code>config.properties</code>
Excel library	Apache POI
Excel organization	One sheet per application module
Excel access	Centralized <code>ExcelReader</code>
DataProvider	Centralized reusable DataProvider
Sheet selection	DataProvider parameter

Area	Your choice
Dynamic data	Custom <code>TestDataGenerator</code>
Selenium utilities	Component-specific utilities/classes
Locators	Page/component-owned <code>By</code> locators
Reporting	ExtentReports
Report	One report per execution
Screenshots	Failure only
Exceptions	Custom exceptions + centralized handling
Logging	ExtentReports only
Test organization	By application module
Test selection	TestNG groups
Test metadata	Description + Test Case ID
Setup/teardown	<code>BaseTest</code>
Browser configuration	<code>config.properties</code>
Configuration access	<code>ConfigReader</code>
Parallel execution	No
CI/CD	Azure DevOps
Execution configuration	<code>config.properties</code>
Retry	Not explicitly answered

One important point: you said these are all the requirements you need, so I will **not introduce a retry mechanism**. The framework will therefore default to **no automatic retry**.

2. Recommended Architecture

The framework will follow this high-level flow:



Excel → ExcelReader → DataProvider → Test Method

↓
TestDataGenerator

config.properties → ConfigReader

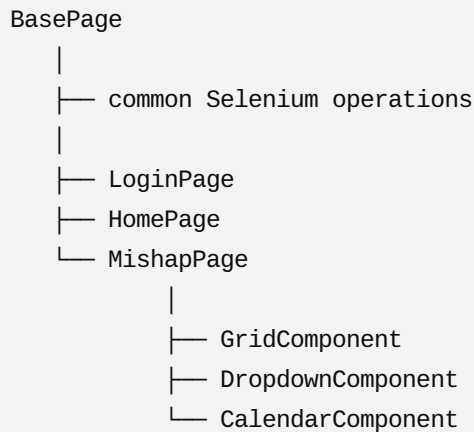
↓
Framework Configuration

Test Failure → Screenshot → ExtentReports

3. Why These Design Decisions?

BasePage + Page Objects + Components

Instead of putting everything into Page Objects:



This prevents duplicated Selenium code.

For example, a grid used on five pages should not require five separate implementations.

Explicit waits only

We won't use:

Java

```
driver.manage().timeouts().implicitlyWait(...)
```

Instead:

Java

```
wait.until(ExpectedConditions.visibilityOfElementLocated(locator));
```

This makes synchronization intentional and avoids the unpredictable behavior that can occur when implicit and explicit waits are mixed.

One WebDriver per test

Your requirement is:

```
Test 1 → Chrome → close
Test 2 → Chrome → close
```

This provides strong test isolation.

Excel + DataProvider

The flow will be:

```
Excel
  ↓
ExcelReader
  ↓
Central DataProvider
  ↓
@Test
```

Your tests won't contain Excel-reading code.

4. Recommended Maven Project Structure

```
selenium-automation-framework/
|
├─ pom.xml
|
├─ src/
|   |
|   └─ main/
|       |
|       └─ java/
|           |
|           └─ com.company.automation/
|               |
|               └─ base/
|                   └─ BasePage.java
|               |
|               └─ components/
|                   └─ GridComponent.java
|                   └─ DropdownComponent.java
|                   └─ CalendarComponent.java
|               |
|               └─ pages/
|                   └─ LoginPage.java
|                   └─ HomePage.java
|                   └─ MishapReportPage.java
|               |
|               └─ driver/
|                   └─ DriverManager.java
```

```
|
|
|   └─ exceptions/
|       └─ FrameworkException.java
|       └─ ElementNotFoundException.java
|
|   └─ utils/
|       └─ ConfigReader.java
|       └─ ExcelReader.java
|       └─ TestDataGenerator.java
|
└─ test/
    └─ java/
        └─ com.company.automation.tests/
            └─ base/
                └─ BaseTest.java
            └─ login/
                └─ LoginTests.java
            └─ mishap/
                └─ MishapReportTests.java
            └─ workspace/
                └─ WorkspaceTests.java
            └─ dataproviders/
                └─ TestDataProvider.java
        └─ resources/
            └─ config.properties
            └─ testdata/
                └─ Login.xlsx
                └─ Mishap.xlsx
                └─ Workspace.xlsx
            └─ extent-config.xml
    └─ test-output/
```

5. Maven Dependencies

Your `pom.xml` will contain the core dependencies:

XML

```

<dependencies>

  <!-- Selenium -->
  <dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>4.x.x</version>
  </dependency>

  <!-- TestNG -->
  <dependency>
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>7.x.x</version>
    <scope>test</scope>
  </dependency>

  <!-- Apache POI -->
  <dependency>
    <groupId>org.apache.poi</groupId>
    <artifactId>poi-ooxml</artifactId>
    <version>5.x.x</version>
  </dependency>

  <!-- ExtentReports -->
  <dependency>
    <groupId>com.aventstack</groupId>
    <artifactId>extentreports</artifactId>
    <version>5.x.x</version>
  </dependency>

</dependencies>

```

I would pin the **current compatible versions** when implementing the actual project rather than blindly using placeholder versions above.

6. Configuration

Your:

```
src/test/resources/config.properties
```

could look like:

```

properties

browser=chrome

```

```
baseUrl=https://your-application-url.com

username=your_username
password=your_password

explicitWait=15

screenshotOnFailure=true

extentReportPath=test-output/ExtentReport.html
```

`ConfigReader` becomes the single access point:

Java

```
String url = ConfigReader.getProperty("baseUrl");
```

This prevents different classes from independently loading properties files.

For a real production/enterprise repository, I would strongly recommend moving passwords out of Git into Azure DevOps secret variables. However, your explicit requirement was to keep them in `config.properties`, so the framework design follows that choice.

7. BasePage

The Base Page should contain **generic UI interaction**, not business logic.

Example:

Java

```
public class BasePage {

    protected WebDriver driver;
    protected WebDriverWait wait;

    public BasePage(WebDriver driver) {
        this.driver = driver;

        this.wait = new WebDriverWait(
            driver,
            Duration.ofSeconds(
                Integer.parseInt(
                    ConfigReader.getProperty("explicitWait")
                )
            )
        );
    }
}
```



```

    }

    protected void click(By locator) {

        WebElement element =
            wait.until(ExpectedConditions.elementToBeClickable(locator));

        element.click();
    }

    protected void type(By locator, String value) {

        WebElement element =
            wait.until(ExpectedConditions.visibilityOfElementLocated(locator));

        element.clear();
        element.sendKeys(value);
    }

    protected String getText(By locator) {

        WebElement element =
            wait.until(ExpectedConditions.visibilityOfElementLocated(locator));

        return element.getText();
    }
}

```

The Page Object then becomes cleaner:

Java

```

public class LoginPage extends BasePage {

    private By username =
        By.id("username");

    private By password =
        By.id("password");

    private By loginButton =
        By.id("loginButton");

    public LoginPage(WebDriver driver) {
        super(driver);
    }

    public LoginPage enterUsername(String usernameValue) {
        type(username, usernameValue);
        return this;
    }
}

```

```

    public LoginPage enterPassword(String passwordValue) {
        type(password, passwordValue);
        return this;
    }

    public HomePage clickLogin() {
        click(loginButton);
        return new HomePage(driver);
    }
}

```

This follows a useful POM principle:

Page Objects should expose business actions rather than raw Selenium implementation details.

8. Component Objects

Suppose the application has the same grid on many pages.

Instead of:

Java

```

MishapPage
WorkspacePage
SearchPage

```

each implementing its own grid logic, create:

Java

```

public class GridComponent {

    private WebDriver driver;
    private WebDriverWait wait;

    private By rows = By.cssSelector("table tbody tr");

    public GridComponent(WebDriver driver) {
        this.driver = driver;
        this.wait = new WebDriverWait(
            driver,
            Duration.ofSeconds(15)
        );
    }

    public int getRowCount() {

```

```

        return wait.until(
            ExpectedConditions
                .presenceOfAllElementsLocatedBy(rows)
        ).size();
    }
}

```

Then:

Java

```

public class MishapReportPage extends BasePage {

    private GridComponent mishapGrid;

    public MishapReportPage(WebDriver driver) {
        super(driver);
        mishapGrid = new GridComponent(driver);
    }

    public int getMishapCount() {
        return mishapGrid.getRowCount();
    }
}

```

This is one of the strongest scalability decisions in your framework.

9. ExcelReader

Apache POI will be isolated inside one utility.

Conceptually:

Java

```

public class ExcelReader {

    public static Object[][] getExcelData(
        String filePath,
        String sheetName) {

        // Open workbook
        // Read sheet
        // Read rows and columns
        // Convert cells to Java values
        // Return Object[][]
    }
}

```

Your test will never need to know:

Java

```
XSSFWorkbook  
XSSFSheet  
XSSFRow  
XSSFCell
```

That implementation stays inside `ExcelReader` .

10. Centralized DataProvider

Example:

Java

```
public class TestDataProvider {  
  
    @DataProvider(name = "excelData")  
    public static Object[][] getExcelData(  
        String sheetName) {  
  
        return ExcelReader.getExcelData(  
            "src/test/resources/testdata/Mishap.xlsx",  
            sheetName  
        );  
    }  
}
```

A test can then consume it:

Java

```
@Test(  
    dataProvider = "excelData",  
    dataProviderClass = TestDataProvider.class,  
    description = "Create a valid mishap report",  
    groups = {"regression"}  
)  
public void createMishapReport(  
    String firstName,  
    String lastName,  
    String companyName) {  
  
    // Test implementation  
}
```

This gives you:

```
Excel
↓
ExcelReader
↓
DataProvider
↓
TestNG
↓
Test
```

11. Dynamic Test Data

Your custom generator could provide:

Java

```
public class TestDataGenerator {

    public static String generateUniqueName() {
        return "TestUser_" + System.currentTimeMillis();
    }

    public static String generateEmail() {
        return "test_" +
            System.currentTimeMillis() +
            "@example.com";
    }

    public static String generatePhoneNumber() {
        return "9" +
            String.valueOf(
                System.currentTimeMillis()
            ).substring(4);
    }
}
```

Then Excel can provide static values while Java creates unique values.

For example:

```
Excel:
Company = ABC Corporation

Generated:
```

12. BaseTest

Because you selected `BaseTest`, the test classes remain clean.

Conceptually:

Java

```
public class BaseTest {

    protected WebDriver driver;

    @BeforeMethod
    public void setUp() {

        driver = new ChromeDriver();

        driver.manage().window().maximize();

        driver.get(
            ConfigReader.getProperty("baseUrl")
        );
    }

    @AfterMethod
    public void tearDown() {

        if (driver != null) {
            driver.quit();
        }
    }
}
```

The final implementation can centralize the lifecycle further while still keeping `BaseTest` as the test-facing entry point.

13. Test Class

A typical test should become very small:

Java

```
public class LoginTests extends BaseTest {
```

```

@Test(
    description = "Verify successful login",
    groups = {"smoke", "regression"}
)
public void verifySuccessfulLogin() {

    LoginPage loginPage =
        new LoginPage(driver);

    HomePage homePage =
        loginPage
            .enterUsername(
                ConfigReader.getProperty("username")
            )
            .enterPassword(
                ConfigReader.getProperty("password")
            )
            .clickLogin();

    Assert.assertTrue(
        homePage.isDisplayed()
    );
}
}

```

Notice what isn't present:

```

WebDriverWait
Excel code
ChromeDriver setup
Screenshot code
ExtentReports code
Configuration loading

```

That's exactly what we want.

14. ExtentReports Architecture

We'll have one report:

```

test-output/
  ExtentReport.html

```

Execution:

```

ExtentReport
|
├─ LoginTests

```

```
|
|   ├── verifySuccessfulLogin
|   └── verifyInvalidLogin
|
|   ├── MishapReportTests
|   |   ├── createMishap
|   |   └── validateMishap
|   └── WorkspaceTests
|       ├── validateGrid
|       └── validateSearch
```

On failure:

```
Test Failure
  ↓
Capture Screenshot
  ↓
Attach Screenshot
  ↓
Mark Extent test Failed
```

Since you selected **failure-only screenshots**, successful tests won't generate unnecessary screenshots.

15. TestNG Groups

Your tests can be categorized:

Java

```
@Test(groups = {"smoke"})
```

or:

Java

```
@Test(groups = {"regression"})
```

or:

Java

```
@Test(groups = {"smoke", "regression"})
```

This allows:

Smoke execution
↓
Smoke tests only

and:

Regression execution
↓
Regression tests

without changing test code.

16. Custom Exceptions

The framework can define:

Java

```
public class FrameworkException
    extends RuntimeException {

    public FrameworkException(String message) {
        super(message);
    }

    public FrameworkException(
        String message,
        Throwable cause) {

        super(message, cause);
    }
}
```

Specialized exceptions can extend it:

Java

```
public class ElementNotFoundException
    extends FrameworkException {

    public ElementNotFoundException(
        String message) {

        super(message);
    }
}
```

This gives errors more meaningful context than simply exposing a low-level Selenium exception.

17. Azure DevOps Flow

Your eventual pipeline will look like:

```
graph TD; A[Developer pushes code] --> B[Azure DevOps Pipeline]; B --> C[Checkout repository]; C --> D[Install Java]; D --> E[Maven dependency resolution]; E --> F[mvn test]; F --> G[TestNG execution]; G --> H[Chrome]; H --> I[ExtentReport]; I --> J[Publish test artifacts];
```

Because the framework uses Maven, Azure DevOps doesn't need to understand your internal framework architecture.

It simply executes the project.

18. Important Design Principle

The most important architectural separation will be:

```
graph TD; A[TEST LAYER] --> B[PAGE LAYER]; B --> C[COMPONENT LAYER]; C --> D[SELENIUM];
```

and separately:

```
TEST
↓
DATAPROVIDER
↓
EXCELREADER
↓
EXCEL
```

and:

```
FRAMEWORK
↓
CONFIGREADER
↓
config.properties
```

This prevents your tests from becoming tightly coupled to implementation details.

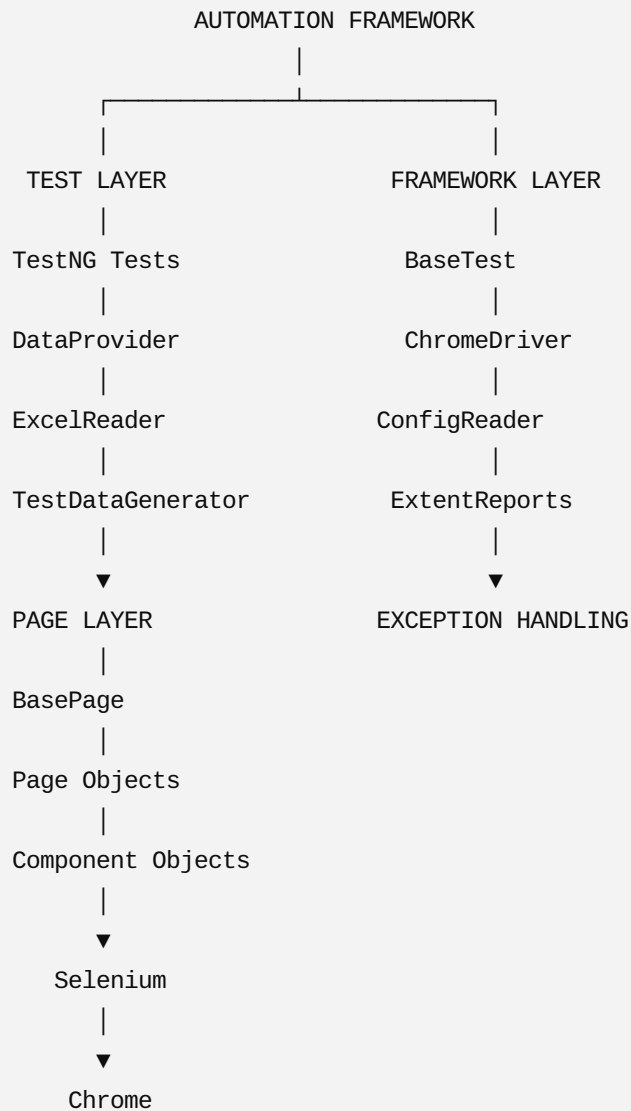
19. What We Should Avoid

Based on your requirements, I would deliberately avoid:

- ✗ Implicit waits
- ✗ Mixing implicit + explicit waits
- ✗ Static/global WebDriver
- ✗ One browser for entire suite
- ✗ Selenium code directly inside tests
- ✗ Excel code directly inside tests
- ✗ Duplicate component logic in pages
- ✗ `Thread.sleep()` as synchronization
- ✗ Parallel execution
- ✗ Multiple reporting frameworks
- ✗ PageFactory
- ✗ Unnecessary abstraction layers
- ✗ Hard-coded test data throughout tests

20. Final Architecture

The resulting framework will essentially be:



This is a **clean, maintainable starting architecture** that is intentionally not over-engineered. It satisfies your current requirements while leaving room to add browsers, additional environments, API integration, parallel execution, secret management, or additional reporting later without redesigning the entire test layer.

The next implementation phase should be done **incrementally**: first Maven/project setup → dependencies → configuration → BaseTest/WebDriver → BasePage → components → ExcelReader → DataProvider → Page Objects → tests → ExtentReports → exception handling → Azure DevOps pipeline.